Code Deep Dive — Every File, Every Concept (Beginner → Confident)

This is the "read it like a book" document. It explains **what each file is, what the important lines mean, and why they exist**, with analogies and senior advice. Read it top to bottom once, slowly. Then keep it open beside you while you code.

Companion docs:

- `docs/BACKEND_GUIDE.md` — the mental model & tour.
- `docs/BUILD_LOG_AND_TESTING.md` — what we built + how to test endpoints.
- `docs/FRONTEND_ROADMAP.md` — how to build the React frontends in phases.

Part 0 — The plumbing (before any Python)

0.1 `requirements.txt` — the shopping list

```
fastapi==0.142.2
uvicorn==0.54.0
supabase==2.32.0
python-dotenv==1.2.4
httpx==0.28.1
resend==2.49.1
pydantic==2.13.5
python-multipart==0.0.32
email-validator==2.3.0
...
```

Each line is `package==exact-version`. Pinning versions means "everyone who installs this gets the **same** code I tested", so it can't break because a library changed overnight.

- **Why so many lines?** `pip freeze` lists **direct** dependencies *and* their *transitive* dependencies (packages your packages need). `fastapi` pulls in `starlette` and `pydantic`; `supabase` pulls in `httpx`, `postgrest`, etc.
- **Important ones for you:**
  - `fastapi` — the web framework.
  - `uvicorn` — the server that runs it.
  - `supabase` — database/auth/storage client.
  - `python-dotenv` — reads `.env` into environment variables.
  - `httpx` — async HTTP client (Daraja + OpenRouter calls).
  - `resend` — sends receipt emails.
  - `pydantic` — data validation models.
  - `python-multipart` — lets FastAPI accept file uploads (product images).
  - `email-validator` — powers `EmailStr`.

Install everything at once (from the project root):

```
# 1. create + activate a virtual environment (isolated package space)
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

# 2. install EXACTLY the pinned versions in one shot
pip install -r requirements.txt

# 3. run the API
uvicorn app.main:app --reload
```

> `-r` means "requirements file". One command installs all 45 packages. A **virtual environment** is a private folder of packages so this project's `fastapi` version doesn't clash with another project's.

Senior advice: never edit `requirements.txt` by hand to add a normal package; run `pip install <pkg>` then `pip freeze > requirements.txt`. And always commit it — "works on my machine" is usually a missing pin.

0.2 `.env` and `.env.example` — secrets vs documentation

`.env` holds real secrets (database keys, API tokens) and is **git-ignored**. `.env.example` holds the **same variable names with fake values** and *is* committed, so a new dev knows what to fill in.

```
cp .env.example .env   # then edit .env with real values
```

Code reads them with `os.environ.get("NAME")`. Think of `.env` as the restaurant manager's key ring — never left on the counter (repo).

0.3 `.gitignore` — what Git must never upload

```
venv/            # local packages
.env             # secrets
__pycache__/     # compiled Python
.env.*           # other env files
!.env.example    # EXCEPT the example (the "!" is a negation)
```

Senior advice: committing a secret key is one of the most common junior mistakes. If you ever do, rotate the key immediately — deleting the file is not enough, it lives in Git history.

Part 1 — The app core (`app/`)

1.1 `app/main.py` — the front door & the switchboard

```
app = FastAPI(title="GWV-xuk API")          # create the application object
```

- `FastAPI(...)` builds a web app. The object is named `app`; `uvicorn app.main:app` means "in file `app/main.py`, run the object called `app`".
- `title=` shows up in the auto-generated `/docs` page.

```
_origins = [o.strip() for o in os.environ.get("CORS_ORIGINS", "").split(",") if o.st
if not _origins:
    _origins = ["http://localhost:3000", "http://localhost:5173"]

app.add_middleware(CORSMiddleware, allow_origins=_origins, allow_credentials=True,
                   allow_methods=["*"], allow_headers=["*"])
```

- This is a **list comprehension**: "split the env var on commas, trim spaces, keep non-empty". `_origins` becomes e.g. `["https://x-uk.com", "http://localhost:5173"]`.
- CORS = a browser rule. Without it, JS on `localhost:5173` cannot call the API on `localhost:8000`. Middleware is code that wraps **every** request.
- **Security note:** never combine `allow_origins=["*"]` with `allow_credentials=True`.

```
app.include_router(products.router)
app.include_router(orders.router)
...
app.include_router(admin_router)
app.include_router(whatsapp_webhook_router)
```

- Each `router` is a group of related endpoints. `include_router` plugs them in.
- Analogy: `main.py` is the **switchboard operator** connecting every department.

```
@app.get("/health")
async def health():
    return {"status": "ok"}
```

- A **decorator** (`@app.get`) attaches the function to a URL + method.
- `async def` marks it **asynchronous** — the server can handle other requests while this one waits on the network.
- `/health` is what uptime monitors ping to check the service is alive.

1.2 `app/database.py` — one shared key to the pantry

```
load_dotenv()                                # read .env into the environment
url = os.environ.get("SUPABASE_URL")
key = os.environ.get("SUPABASE_KEY")
if not url or not key:
    raise RuntimeError("Missing SUPABASE_URL or SUPABASE_KEY environment variables")
supabase: Client = create_client(url, key)
```

- **Fail fast:** if secrets are missing, crash at startup instead of failing mysteriously later.
- **One client, reused everywhere.** Creating a new client per request wastes connections. Every other file does `from app.database import supabase`.

Senior advice: this is the **single source of truth** pattern. If you ever change databases, you change one file.

1.3 `app/dependencies.py` — the bouncer and the VIP list

```
security = HTTPBearer()
```

- Tells FastAPI to expect an `Authorization: Bearer <token>` header and to parse it for us.

```
async def get_current_user(credentials = Depends(security)):
    token = credentials.credentials
    try:
        user_response = supabase.auth.get_user(token)   # Supabase validates the JWT
        return user_response.user
    except Exception:
        raise HTTPException(401, "Invalid or expired authentication token")
```

- **Dependency injection:** a route writes `user = Depends(get_current_user)` and FastAPI runs this first, injecting the result.
- `user_response.user` is an object with `.id` and `.email`.
- This is **authentication** = "who are you?".

```
_ADMIN_EMAILS = {e.strip().lower() for e in os.environ.get("ADMIN_EMAILS","").split(

async def require_admin(current_user = Depends(get_current_user)):
    if (current_user.email or "").lower() in _ADMIN_EMAILS:
        return current_user
    res = supabase.table("profiles").select("role").eq("user_id", current_user.id).s
    if isinstance(res.data, dict) and (res.data.get("role") or "").lower() == "admin
        return current_user
    raise HTTPException(403, "Admin privileges required")
```

- `{...}` is a **set** — fast membership tests, no duplicates.
- This is **authorization** = "what may you do?". Two ways to be admin: env allowlist *or* `profiles.role`. `403` means "I know who you are, but you're not allowed"; `401` means "I don't know who you are".

Senior advice: authentication ≠ authorization. The old code confused them: it checked you were logged in, not that you were an admin.

Part 2 — The services layer (`app/services/`) — the kitchen

Routers should be **thin** (HTTP in/out). Real logic lives in services so it can be reused and tested. This is the **service layer** pattern.

2.1 `app/services/inventory.py` — never lose stock

```
STOCK_HELD_STATUSES = {"Pending PIN", "Processing"}
```

- Statuses where we've reserved stock but not yet collected money.

```
def restore_order_stock(order):
    for item in order.get("items", []):        # .get avoids KeyError
        product_id = item.get("id") or item.get("product_id")
        quantity = int(item.get("qty"))
        res = supabase.table("products").select("stock_quantity").eq("id", product_i
        current = int(res.data[0].get("stock_quantity") or 0)
        supabase.table("products").update({"stock_quantity": current + quantity}).ec
```

- Adds each line item's quantity back to the product.

```
def release_stock_once(order, new_status="Payment Failed"):
    if order.get("status") not in STOCK_HELD_STATUSES:
        return False
    res = (supabase.table("orders")
           .update({"status": new_status})
           .eq("id", order["id"])
           .eq("status", current_status)     # <-- only if still in the SAME status
           .execute())
    if not res.data:
        return False                         # someone already did it
    restore_order_stock(order)
    return True
```

- **Idempotency:** payment webhooks get retried. The `.eq("status", current)` guard means the *database* only lets **one** caller flip the order, so stock is restored exactly once. Two people reaching for the same cinema seat — the ticket machine prints only one.

2.2 `app/services/mpesa.py` — one brain for payment results

```
def process_stk_callback(payload):
    body = payload.get("Body", {}).get("stkCallback", {})
    checkout_request_id = body.get("CheckoutRequestID")
    result_code = body.get("ResultCode")
    order = ...find order by checkout_request_id...
    if result_code == 0:            # success
        receipt = ...MpesaReceiptNumber...
        if order and order["status"] != "Paid":
            mark Paid + save receipt + email
    else:
        new_status = "Cancelled" if result_code == 1032 else "Payment Failed"
        release_stock_once(order, new_status)
```

- `payload.get("Body", {}).get("stkCallback", {})` safely walks nested JSON without crashing if a level is missing.
- `1032` = the customer pressed cancel on their phone. Any other non-zero = failed.
- Both webhook URLs call this one function, so there's no second copy to forget.

2.3 `app/services/ai.py` — one door to the LLM

```
async def chat_completion(messages, model=DEFAULT_MODEL, temperature=0.7, max_tokens
    payload = {"model": model, "messages": messages, "temperature": temperature, "ma
    async with httpx.AsyncClient() as client:
        response = await client.post(OPENROUTER_URL, headers={"Authorization": f"Bea
    return response.json()["choices"][0]["message"]["content"]
```

- `messages` is a list of `{"role": "user"/"assistant"/"system", "content": "..."}`.
- `temperature` = randomness (0 = deterministic, 1 = creative).
- `async with` opens/closes the HTTP client cleanly (a **context manager**).

2.4 `app/services/whatsapp.py` — the triage bot

Key pieces:

```
async def send_whatsapp_message(to, body):
    payload = {"messaging_product": "whatsapp", "to": to, "type": "text", "text": {'
    ... POST to graph.facebook.com/{GRAPH_VERSION}/{PHONE_ID}/messages ...
```

- Sends a WhatsApp message through Meta's Cloud API.

```
def extract_inbound_message(payload):
    value = payload["entry"][0]["changes"][0]["value"]
    message = value["messages"][0]
    ...
```

- Meta nests messages deeply. This digs out `from`, `text`, `id`, and the contact's `name`.

```
def get_or_create_lead(phone, name=None):
    res = supabase.table("leads").select("*").eq("phone", phone).execute()
    if res.data: return existing
    return insert new lead
```

- **Upsert-by-hand:** find the lead by phone or create it.

```
def _status_from_score(score):
    if score >= HOT_THRESHOLD: return "hot"
    if score >= WARM_THRESHOLD: return "warm"
    return "cold"
```

- Turns a 0–100 AI score into a human label.

```
async def handle_inbound_message(payload):
    ...
    raw = await chat_completion(messages, temperature=0.4, max_tokens=500)
    reply, meta = _parse_model_output(raw)      # expects JSON {reply, score, ...}
    log_message(lead_id, "outbound", reply)
    updated = _apply_meta(lead_id, meta)
    await send_whatsapp_message(phone, reply)
    if updated.get("status") == "hot" and not was_hot:
        await _notify_owner(lead, meta)
```

- Wrapped in `try/except` because it runs as a **background task** — a crash must never kill the worker.
- `_parse_model_output` tolerates the model wrapping JSON in ```` ```json ```` fences, which LLMs love to do.

Senior advice: LLM output is never guaranteed valid. Always parse defensively and have a fallback (here, send the raw text).

Part 3 — The routers (`app/routers/`) — the waiters

Every router follows the same skeleton:

```
router = APIRouter(prefix="/x", tags=["X"])   # URL group + docs category

class SomeModel(BaseModel):                    # request/response shape (Pydantic)
    field: type

@router.get("/path")                           # decorator = method + URL
def handler(arg, user = Depends(get_current_user)):
    ... use supabase ...
    return {"success": True, "data": ...}
```

3.1 `auth.py` — the signup/login desk

- `SignUpRequest` / `LoginRequest` are Pydantic models. `EmailStr` auto-validates emails (rejecting `"not-an-email"` with `422`).
- `supabase.auth.sign_up(...)` creates a user; `sign_in_with_password(...)` returns a `session` containing the `access_token` (JWT) and `refresh_token`.
- New: `GET /me`, `POST /refresh` (swap refresh token for a new session), `POST /logout` (revoke), `POST /forgot-password` (always returns success so attackers can't probe which emails exist — a real security practice).

3.2 `products.py` — the catalogue

```
query = supabase.table("products").select("*")
if search:   query = query.ilike("name", f"%{search}%")
if category: query = query.eq("category", category)
query = query.order(sort_field, desc=is_desc)
response = query.execute()
```

- **Builder pattern:** each filter conditionally adds a clause; nothing runs until `.execute()`.
- `ilike` = case-insensitive LIKE; `%` are wildcards (`%iphone%`).
- Write routes (`POST/PATCH/DELETE`) carry `Depends(require_admin)`.

3.3 `categories.py` — public storefront categories

- One endpoint: `GET /categories/` returns `active = true` rows ordered by name.
- Deliberately **public** so visitors can browse navigation before signing up.

3.4 `cart.py` — shopping cart

- `POST /cart/add` checks if the row exists → increments, else inserts.
- `PATCH /{item_id}` sets an exact quantity and refuses to exceed stock.
- `DELETE /{item_id}` removes a line; `DELETE /` clears everything.
- **All queries filter by `.eq("user_id", current_user.id)`** so you can only touch your own cart (the IDOR fix).

3.5 `wishlist.py` — saved items

- Same ownership pattern. `DELETE /{product_id}` removes by product id so the frontend can build a simple heart-toggle.

3.6 `reviews.py` — verified-buyer reviews

- Before inserting, it loads the user's orders and checks the product id appears in some

order's `items` JSON.

- `Field(..., ge=1, le=5)` forces the rating to be between 1 and 5.
- User id comes from the token, never the body.

3.7 `profiles.py` — profile

- `GET/POST/PATCH /profiles/me` . `payload.dict(exclude_unset=True)` keeps only fields the client actually sent, so a partial update doesn't wipe other data.
- `date` is converted to ISO text because JSON has no date type.

3.8 `orders.py` — checkout, tracking, admin actions

The checkout (`process_checkout`) is the crown jewel:

```
for item in payload.items:
    product_row = ...select("name, price, stock_quantity")...
    if validated_stock < item.qty: raise 400
    unit_price = float(product_row["price"])        # PRICE FROM DB
    items_total += unit_price * item.qty
order_total = round(items_total + delivery_cost, 2)
```

- **Never trust the client.** The client's `price` / `total` are ignored; the total is computed from the database.
- Order is saved first, stock decremented with rollback, then STK push; if payment can't start, stock is released and the order marked failed.
- `"user_id": current_user.id` ties the order to the logged-in buyer (so `/orders/me` works and guests must sign up to pay).
- `POST /{order_id}/cancel` lets a customer cancel their own order and restores stock once.

3.9 `payments.py` — the webhook

```
@router.post("/callback")
async def mpesa_callback(request: Request):
    payload = await request.json()
    process_stk_callback(payload)
    return {"ResultCode": 0, "ResultDesc": "Accepted"}
```

- Thin adapter over `services/mpesa.py` . `Request` gives us the raw body.

3.10 `delivery.py` — locations

- `GET /delivery/locations` returns active locations + cost (used at checkout).

3.11 `upload.py` — avatars

- Validates the file type/extension, generates a UUID filename, uploads to the `profile-assets` bucket under the user's own folder (`f"{current_user.id}/..."`), returns the public URL.

3.12 `analytics.py` — legacy admin summary

- Computes daily revenue / pending orders / low stock. Superseded by `/admin/dashboard/`

`summary` but kept for compatibility.

3.13 `ai.py` — shopping assistant

- Loads products, builds a system prompt, calls OpenRouter, stores the chat in `chat_history`. Uses `Depends(get_current_user)` so conversations are per-user.

3.14 `daraja_service.py` — not a router!

- A **helper class** (note: `class DarajaService:`), not an `APIRouter`. It never appears in routes. It:
  - fetches an OAuth token (`get_access_token`),
  - builds the STK push password from shortcode+passkey+timestamp,
  - POSTs the payment request, and
  - can query transaction status.
- **Name matters:** things inside `routers/` are *not* automatically endpoints; only objects registered with `include_router` are.

Part 4 — The admin package (`app/routers/admin/`)

4.1 `admin/__init__.py` — one lock for the whole wing

```
router = APIRouter(prefix="/admin", tags=["Admin"],
                   dependencies=[Depends(require_admin)])
router.include_router(dashboard.router)
...
```

- `dependencies=[...]` on the **parent** router runs `require_admin` for **every** child route. You cannot forget it. This is the guard-once pattern.

4.2 The modules

- `dashboard.py` — headline numbers (revenue, orders, low stock, customers).
- `orders.py` — paginated list (`page_range`), filter, status update; cancel routes through `release_stock_once`.
- `products.py` — list/update/delete + image add/remove (with storage cleanup).
- `categories.py` — CRUD with auto-slug (`re.sub(r"[^a-z0-9]+", "-", name)`).
- `customers.py`, `inventory.py`, `delivery.py`, `reviews.py`, `leads.py`.

Common patterns to notice (study these!):

- `count="exact"` + `.range(start, end)` = server-side pagination.
- `.or_(f"name.ilike.%{term}%,phone.ilike.%{term}%")` = search across columns.
- `model.dict(exclude_unset=True)` = safe partial updates.
- Ownership/role checks happen **before** any write.

Part 5 — Webhooks (`app/routers/webhooks/`)

`whatsapp.py`

```
@router.get("")
def verify_webhook(hub_mode = Query(None, alias="hub.mode"), ...):
    if hub_mode == "subscribe" and hub_verify_token == WHATSAPP_VERIFY_TOKEN:
        return PlainTextResponse(hub_challenge)
    return PlainTextResponse("Forbidden", 403)
```

- Meta calls `GET` once with `hub.*` query params. `alias="hub.mode"` maps the dotted name to a Python-friendly variable. We echo back the `challenge`.

```
@router.post("")
async def receive_webhook(request: Request, background_tasks: BackgroundTasks):
    payload = await request.json()
    background_tasks.add_task(handle_inbound_message, payload)
    return {"status": "received"}
```

- **Ack fast, work later.** Meta expects a quick 200. The LLM reply is slow, so we queue it with `BackgroundTasks` and return immediately.

Part 6 — Utilities (`app/utils/`)

`pagination.py`

```
def page_range(page, page_size):
    page = max(1, page)
    page_size = min(max(1, page_size), MAX_PAGE_SIZE)   # clamp to 100
    start = (page - 1) * page_size
    return start, start + page_size - 1
```

- Converts a friendly `page=2, page_size=20` into the `start` / `end` rows a DB wants, and prevents a client from asking for 1,000,000 rows.

`email.py`

```
resend.api_key = os.environ.get("RESEND_API_KEY")
def send_receipt_email(to_email, order_reference, total, items):
    items_html = "".join([f"<li>{item['name']} ...</li>" for item in items])
    resend.Emails.send({...})
```

- Builds simple HTML and sends. Wrapped in try/except so a failed email never breaks a successful order (**never let a side effect fail the main job**).

Part 7 — The `tests/` folder

```
tests/
├── simulate_mpesa_webhook.py    # manual M-Pesa callback simulator (not pytest)
├── conftest.py                  # pytest setup: dummy env + `client` fixture
├── fakes.py                     # in-memory fake Supabase
├── test_pagination.py           # page_range() clamping
├── test_inventory.py            # stock restore correctness + idempotency
├── test_whatsapp_helpers.py     # scoring, JSON parsing, signature verification
└── test_security_boundaries.py  # guests blocked from protected routes
```

Two different things live here:

1. **`simulate_mpesa_webhook.py`** — a **manual helper**. It fakes Safaricom so you can trigger the payment callback on demand (run it yourself while the server is up). It just sends a request and prints the result; it doesn't assert anything.

2. **`test_*.py`** — real **automated tests** run by `pytest`. They arrange some state, call code, and **assert** the outcome. They run in ~1 second with no network because `fakes.py` replaces Supabase.

Key ideas to learn here:

- `conftest.py` sets dummy env vars *before* importing the app, then defines a `client` fixture. Any test that asks for `client` gets it injected.
- `fakes.py` mirrors the app's chained DB calls (`.table().select().eq().execute()`) using a plain dict, so no database is needed.
- `monkeypatch.setattr(module, "supabase", fake)` swaps the real client for the fake **inside one test only**.

Run them:

```
pip install -r requirements.txt -r requirements-dev.txt
pytest            # all 25
pytest -v         # verbose
pytest tests/test_inventory.py -v
```

See `docs/BUILD_LOG_AND_TESTING.md` §9 for the full "how to use / how to add a test" cheat-sheet. In short: tests are your safety net; without them every edit is a gamble.

Part 8 — The SQL files (`supabase/`)

8.1 `migrations/001_create_categories.sql`

```
create table if not exists public.categories (
    id uuid primary key default gen_random_uuid(),
    name text not null unique,
    slug text not null unique,
    ...
);
alter table public.products add column if not exists category_id uuid references pul
```

- `uuid` primary key with a default generator; `unique` prevents duplicates; `on delete set null` means deleting a category doesn't delete products.

8.2 `migrations/002_add_profiles_role.sql`

- Adds `profiles.role` with a `check (role in (...))` constraint — the DB refuses typos like `'admni'`. This is the column `require_admin` reads.

8.3 `migrations/003_create_leads.sql`

- Creates `leads`, `whatsapp_messages`, `lead_events`, plus a **trigger** that auto-updates `updated_at` on every change. A trigger is SQL that runs itself.

8.4 `migrations/004_rls_new_tables.sql` and `rls_policies.sql`

- **RLS = Row Level Security.** Even with the API's checks, someone with the public key could hit the database directly. RLS is rules *inside the database*:

  ```
  create policy "cart_owner_all" on public.cart_items
      for all using (auth.uid() = user_id) with check (auth.uid() = user_id);
  ```

  `auth.uid()` is the logged-in user id. This says "you may only touch your own cart rows".
- `is_admin()` is a `security definer` function (it runs with elevated rights so it can read `profiles` regardless of RLS).

Senior advice: API checks protect you *through the API*. RLS protects the data itself. Use both — **defense in depth**.

Part 9 — How it all connects (trace one purchase)

```
React "Buy" button
  → POST /orders/checkout + Bearer token
    → CORSMiddleware allows the origin
    → FastAPI validates CheckoutRequest (Pydantic)
    → get_current_user() validates the JWT (dependencies.py)
    → process_checkout() computes total from DB prices (orders.py)
    → services/inventory? no decrement happens inline with rollback
    → DarajaService.send_stk_push() → phone prompt
  → Safaricom POST /payments/callback
    → services/mpesa.process_stk_callback
      → success: mark Paid + email
      → fail: services/inventory.release_stock_once (restore)
```

Every arrow is a file we covered. That's the whole backend.

Part 10 — How to become a better dev (mentor notes)

1. **Read error tracebacks bottom-up.** The last line is the cause; the lines above show the path.
2. **Change one thing, then test.** Big batches hide which change broke it.
3. **Name things honestly.** `release_stock_once` tells you it's idempotent.
4. **Trust the database, not the client.** Every security bug we fixed came from trusting user input.
5. **Write a test for every bug you fix** so it can't return.

6. **Comment the *why*, not the *what*.** Code says what; comments explain why.
7. **Learn `git` well** (branch, diff, stash) — it's your time machine.
8. **Read your own code tomorrow.** If it confuses you then, fix the names.
9. **Prefer boring, obvious code.** Clever code is expensive to maintain.
10. **Build small, ship early, iterate.** Working beats perfect.

How to learn this codebase yourself: open `/docs` and call every endpoint in Swagger, then read the router for each. Trace one request from `main.py` to the database. Re-implement one endpoint from scratch without looking. That loop is how fluency is built.

Next: read `docs/FRONTEND_ROADMAP.md` for how to build the React apps in phases.